Análisis de Caso

Arquitecturas de Almacenamiento de Objetos en la Nube

Eva Durán

alkemy

Análisis de Caso

Arquitecturas de Almacenamiento de Objetos en la Nube

Situación inicial 📍

Una empresa tecnológica en crecimiento, llamada **MediaStream**, especializada en distribución de contenido multimedia (videos educativos, podcast, cursos online) ha experimentado un gran aumento en el tráfico y la demanda de sus servicios a nivel internacional. Actualmente, sus archivos multimedia están almacenados en servidores locales, lo que genera cuellos de botella en el acceso desde otras regiones, problemas de disponibilidad y costos crecientes de mantenimiento.

La dirección de TI propone migrar todo el almacenamiento a una solución basada en la nube, utilizando una arquitectura de almacenamiento de objetos que permita mayor escalabilidad, distribución geográfica y entrega eficiente de contenido. Sin embargo, el equipo necesita definir cómo será la arquitectura, qué servicios usar, cómo configurar el acceso vía URL, cómo gestionar los respaldos, y qué estrategia de distribución será la más eficaz.

Descripción del Caso 🔎

En este contexto, asumirás el **rol de consultor/a en arquitectura cloud**, encargado/a de diseñar y justificar una solución basada en almacenamiento de objetos en la nube, que contemple escalabilidad, seguridad, eficiencia en costos y entrega de archivos multimedia. Deberás analizar proveedores, establecer un esquema de buckets y accesos, y proponer una arquitectura general aplicable a este tipo de negocio digital.

DESARROLLO 📄

1) Diagnóstico inicial

Limitaciones de seguir on-premise (caso MediaStream)

- **Latencia y mala experiencia fuera de la región**: al entregar archivos solo desde un datacenter local, los usuarios de otras zonas geográficas sufren mayores tiempos de descarga. En arquitecturas modernas de objetos, los datos se distribuyen globalmente y se entregan por HTTP(S), lo que justamente ataca este dolor.
- **Escalamiento complejo y costoso**: crecer en on-premise implica comprar y operar más almacenamiento y redes. El almacenamiento de objetos escala horizontalmente sin complejidad de hardware. Seguir on-premise preserva ese cuello de botella.
- **Entrega global ineficiente (sin CDN integrada)**: sin una red de distribución, el tráfico internacional congestiona el backend y encarece el ancho de banda. Los buckets como origen de CDN son el patrón estándar para multimedia a escala. Quedarse on-premise limita (o encarece) ese patrón.
- **Complejidad en continuidad de negocio/Plan de recuperación ante desastres**: replicar datos entre sitios físicos propios es caro y lento. En objetos, la replicación entre zonas/regiones está incorporada para resiliencia y recuperación ante desastres. No adoptar estas medidas mantiene un riesgo operativo mayor.
- **Cargas en el backend de aplicaciones**: con on-premise, típicamente todo pasa por servidores de aplicación. En objetos, los clientes (web/móvil) pueden descargar/subir directo al bucket, reduciendo latencia y tráfico en backend; sin eso, el backend se vuelve cuello de botella.
- **Gestión y costos de ciclo de vida más altos**: sin políticas automáticas es difícil mover contenido "frío" a capas baratas. En el almacenamiento de objetos esto es nativo (versionado y políticas de ciclo de vida), y no aprovecharlo encarece costos manteniéndose on-premise.

En resumen, on-premise limita alcance global, elasticidad, resiliencia y eficiencia de costos que una plataforma de contenidos necesita.

¿Por qué almacenamiento de objetos es el enfoque adecuado?

- **Escalabilidad horizontal y distribución global**: diseñado para volúmenes masivos y acceso mundial, mediante HTTP/S y con replicación entre regiones para baja latencia. Ideal para video, audio y recursos estáticos.

- **Origen natural para CDN**: patrón probado para distribución multimedia. S3 como "origen" y CloudFront para distribución de contenido multimedia.
- **Acceso vía URL (públicas o firmadas):** simplifica la entrega y el compartir seguro con caducidad ("presigned URLs"), sin montar servidores web adicionales.
- **Carga/descarga directa desde clientes**: SDKs oficiales (conjunto de herramientas, librerías y documentación que un proveedor, en esta caso AWS, ofrece para que los desarrolladores puedan conectar fácilmente sus aplicaciones con los servicios de la nube) permiten subir/descargar directo al bucket con control de permisos granulares, descargando al backend de MediaStream.
- **Seguridad y cumplimiento integrados:** cifrado en tránsito y en reposo, políticas de acceso (IAM/RBAC), logs de acceso y auditoría para regulaciones.
- **Durabilidad y Recuperación ante desastres nativos**: replicación multi-zona y alta durabilidad para continuidad del negocio.
- **Optimización de costos por capas**: Políticas de ciclo de vida mueven automáticamente contenido poco accedido a clases de bajo costo (p.ej., Glacier/Archive), manteniendo solo lo más solicitado en estándar.

2) Diseño de arquitectura

2.1 Proveedor elegido

Amazon Web Services (AWS): ofrece servicios de almacenamiento de objetos maduros, acceso por URL, integración nativa con CDN, políticas de ciclo de vida y seguridad granular.

2.2 Estructura del almacenamiento (buckets, carpetas, acceso)

Buckets

- **mediastream-origin** (MAESTROS): privado
  - Contiene los "*masters*" (video/audio/imagen en máxima calidad, archivos orginales).
  - Versionado activado y bloqueo de acceso público.
  - Se usa para ingesta mediante Request Signed URL (PUT prefirmado).
  - Ciclo de vida: STANDARD → STANDAR IA (30–60 días) → Glacier (≥6 meses) → Deep Archive (≥24 meses).
- **mediastream-derived** (DERIVADOS): privado
  - Contiene los archivos "derivados" (copias optimizadas y más livianas, diseñadas para su entrega a los usuarios finales a través de CDN) listos para streaming/distribución.
  - Bloqueo público; lectura solo desde CloudFront por OAC

(Origin Access Control).
  - Ciclo de vida: STANDARD (30–60 días) → Standard-IA (2–6 meses desde último acceso) → Glacier (≥6 meses).

Carpetas / prefijos sugeridos

Usar prefijos (carpetas lógicas) consistentes para mejorar búsquedas, métricas y reglas de ciclo de vida.

```
mediastream-origin/
  video/{cursoId}/{idioma}/master/v{n}/{slug}.mp4
  audio/{podcastId}/master/v{n}/{slug}.wav
  image/{cursoId}/master/{slug}.tif

mediastream-derived/
  video/{cursoId}/{idioma}/hls/{quality}/{segment}.m4s
  video/{cursoId}/{idioma}/mp4/{quality}/{slug}.mp4
  audio/{podcastId}/aac/{bitrate}/{slug}.m4a
  thumbnails/{cursoId}/{slug}.jpg
```

Acceso (público/restringido)

- Buckets: todos privados.
- Lectura pública: no se expone S3. La entrega al usuario final siempre es vía CloudFront.
- Lectura desde CDN: CloudFront accede a mediastream-derived mediante OAC.
- Escritura/ingesta: solo con Request Signed URL (PUT prefirmado) emitida por la API (con Cognito para Autenticación).
- Permisos: IAM con mínimo privilegio (roles separados para API, CDN/OAC y administración).

2.3 Entrega de contenido vía URL

Patrón principal: CDN + URLs de CloudFront

- Los usuarios consumen contenido usando URLs de CloudFront (p. ej., https://cdn.mediastream.com/...).
- El origen de esa distribución es mediastream-derived (privado). CloudFront obtiene los objetos por OAC.
- Para cursos de pago/áreas privadas, usar CloudFront Signed URLs (o Signed Cookies) con expiración para autorizar el acceso por tiempo limitado.
- Para material totalmente público (promos, landing), se puede usar URL directa de CloudFront sin firma.

Subidas de datos a la plataforma

- El cliente solicita a la API una Request Signed URL y sube el archivo directo a mediastream-origin.
- Ventajas: menos carga en el backend, menor latencia y control fino del tiempo/objeto permitido.

Resumen operativo

- **Lectura**: Usuario → CloudFront → (si es necesario) S3 mediastream-derived vía OAC.
- **Escritura**: Usuario → API (Request Signed URL) → S3 mediastream-origin.
- **Protección**: Nada público en S3; control de acceso en el origen (CDN con firmas) y en ingesta (URLs prefirmadas + Cognito).

2.4 Políticas de ciclo de vida y clases de almacenamiento

Aunque se detalla más en el punto 3, es parte del diseño lógico del bucket.

- **Contenido "caliente"** (últimos estrenos): permanecer en Standard/Hot.
- **Catálogo con baja demanda**: pasar a Infrequent Access/Cold tras 30–60 días.
- **Master/source** o materiales históricos: mover a Archive/Glacier a 90–180 días, con retenciones exigidas por negocio.

2.5 Esquema de identidades y permisos

- **IAM roles** por servicio (ingesta, CDN-origin) con principio de menor privilegio.
- **Bloqueo de acceso público** a nivel de bucket junto con políticas que solo permitan "GetObject" (Rol de lectura) desde CloudFront (OAC).
- **KMS** para cifrado de objetos y logs de acceso habilitados (S3/CloudFront).

Resultado esperado del diseño

- **S3** como repositorio de objetos privado y escalable.
- **CloudFront** como único punto de entrega al público, garantizando baja latencia mediante su red de distribución global (edge locations).
- **Cargas** seguras por URLs firmadas.
- **Ciclo de vida** automatizado para optimizar costos en el tiempo.
- **Metadatos y versionado** para gobernanza y trazabilidad.

3) Estrategia de respaldo y recuperación

3.1 ¿Cómo se realizarán los respaldos?

- **Versionado activado en buckets**: cada vez que se modifique o reemplace un archivo multimedia, S3 guardará la versión anterior. Esto protege frente a borrados accidentales o corrupción de archivos.
- **Backups automáticos de logs y metadatos**: los buckets de auditoría tendrán políticas de retención más largas, almacenando registro de acceso para fines legales y de seguridad.

3.2 ¿Qué objetos se archivarán?

- **Contenido "maestro":** Archivos originales, en su máxima calidad (videos en resolución original, audios sin compresión), se almacenarán a largo plazo en clases como S3 Archive, ya que no se consulta frecuentemente.
- **Cursos inactivos o temporadas antiguas:** se moverán tras cierto tiempo a almacenamiento de menor costo, pero conservando acceso si es necesario recuperar el contenido.
- **Miniaturas e índices de catálogos:** se mantienen en estándar porque son consultados todo el tiempo.

3.3 ¿Se usarán diferentes capas de almacenamiento?

Sí, se aplicarán políticas de ciclo de vida para mover automáticamente objetos según su uso:

- **S3 Standard (caliente):**
    - Videos recientes, cursos en curso.
    - Alta disponibilidad y baja latencia.
    - Tiempo de retención: primeros 30-60 días.
- **S3 Infrequent Access (frío):**
    - Contenido menos consultado, pero que aún debe estar accesible bajo demanda.
    - Costos más bajos que Estándar, con un pequeño cargo por recuperación.
    - Retención 2-6 meses después de su último acceso.
- **S3 Glacier/Deep Archive (archivo):**
    - Masters de videos, audios originales y catálogos históricos.
    - Ideal para contenido que rara vez se accede, pero debe preservarse (legales, patrimoniales, backups completos).
    - Costos muy bajos de almacenamiento, con tiempos de recuperación que van desde minutos (Glacier Instant) hasta horas (Deep Archive).

MediaStream tendrá un sistema híbrido de almacenamiento por niveles, donde lo más usado queda en Standard, lo intermedio pasa a Infrequent Access, y lo patrimonial/histórico a Glacier. Todo con versionado más replicación entre regiones, asegurando resiliencia, costos eficientes y recuperación garantizada.

4) Control de Acceso y Seguridad

4.1 Mecanismos para asegurar los datos

1. **IAM (Identity and Access Management):**

Definir roles y políticas de acceso con el principio de menor privilegio:

- Rol de lectura (GetObject) solo para CloudFront (CDN): la idea es que el bucket de S3 no sea público y en lugar de que cualquiera pueda entrar, solo CloudFront tiene permiso para leer los archivos desde el bucket. Los usuarios finales reciben contenido a través de la CDN, no directamente desde S3. Esto protege el bucket y además acelera la distribución.
- Rol de carga (PutObject) solo para aplicaciones autorizadas o presigned URLs: cuando alguien sube un archivo (ej. Un administrador que carga un video nuevo), no debe hacerlo con permisos abiertos. Solo la aplicación backend o URLs firmadas pueden hacerlo. Así se evita que cualquier persona intente subir cosas al bucket.
- Roles separados para administración y auditoría: Administración es el equipo técnico que puede crear buckets, configurar reglas y modificar políticas. Auditoría es el equipo que solo puede leer logs y verificar accesos, pero sin poder cambiar nada. Separar funciones evita errores y malas prácticas, y asegura trazabilidad en quién hace qué.

2. **Cifrado:**

- En reposo: habilitar SSE-KMS (Server-Side Encryption con AWS KMS) para que todos los objetos estén cifrados automáticamente.
- En tránsito: forzar uso de HTTPS/TLS en todas las interacciones con S3 y CloudFront.

3. **Logs y monitoreo:**

- S3 Access Logs: activados en un bucket dedicado.
- CloudFront logs: para analizar patrones de consumo y posibles abusos.
- CloudTrail: para auditar accesos administrativos.
- De manera opcional podría integrarse con Amazon GuardDuty (detección de amenazas) y AWS Config (cumplimiento de políticas).
- CloudWatch: para monitoreo de métricas, alarmas, dashboards y análisis de logs. No genera los logs, sino que los centraliza para alertar y visualizar. Es como una capa de observabilidad.

4. **Protección perimetral:**
- Bucket con Bloqueo de Acceso Público activado.
- Acceso solo desde CloudFront mediante Origin Access Control (OAC).
- AWS WAF (Web Application Firewall) en la distribución CDN para mitigar ataques como DDoS, inyección o bots.

5. **Gestión de versiones y eliminación:**
- Versionado habilitado para poder restaurar objetos borrados/modificados.
- MFA Delete para prevenir borrados accidentales o maliciosos.

4.2 Buenas prácticas recomendadas para el equipo

- **Seguridad por defecto**: nunca habilitar accesos "públicos" al bucket salvo casos muy específicos.
- **Principio de menor privilegio**: asignar a cada rol solo los permisos necesarios, revisando y ajustando periódicamente.
- **Rotación y uso de credenciales temporales:** evitar contraseñas o claves fijas, usar tokens temporales.
- **Monitoreo continuo**: revisar logs de acceso regularmente para detectar patrones anómalos.
- **Automatización de políticas**: se podría aplicar Insfrastructure as Code (CloudFormation/Terraform) para que la configuración de seguridad no dependa de cambios manuales.
- **Pruebas de recuperación y simulacros de incidente**: validar que versionado, plan de recuperación ante desastres y alertas funcionan como se espera.
- **Capacitación del equipo**: capacitar a desarrolladores y administradores en buenas prácticas de seguridad en la nube.

Con esto MediaStream tendrá un entorno cifrado, auditado y gobernado, con acceso mínimo y controlado, y con la trazabilidad necesaria para auditorías o cumplimiento regulatorio.

5) Distribución de contenido y escalabilidad

Mecanismos para mejorar el acceso global

- CDN delante de S3 (origen): Publica los objetos multimedia en buckets y poner una CDN como capa de entrega, en este caso Amazon CloudFront. La CDN cachea cerca del usuario y baja la latencia mundialmente. El bucket queda como "origen" del contenido a distribuir.

- Acceso vía URL (directas o firmadas): El patrón orientado a URL permite exponer archivos con enlaces públicos, prefirmados (que expiran) o protegidos por autenticación. Ideal para multimedia y descargas bajo control.
- Versiones por dispositivo/calidad: Guarda variantes para que el cliente reciba la más adecuada (por ejemplo: resolución de videos, miniaturas y bitrate adaptativo).
- Replicación y alcance global del origen: la distribución/replicación geográfica de objetos más CDN da disponibilidad alta y latencias bajas sin servidores de archivos tradicionales.

Como resultado se espera tiempos de carga menores, menos tráfico al backend y mejor experiencia para imágenes, audio y video.

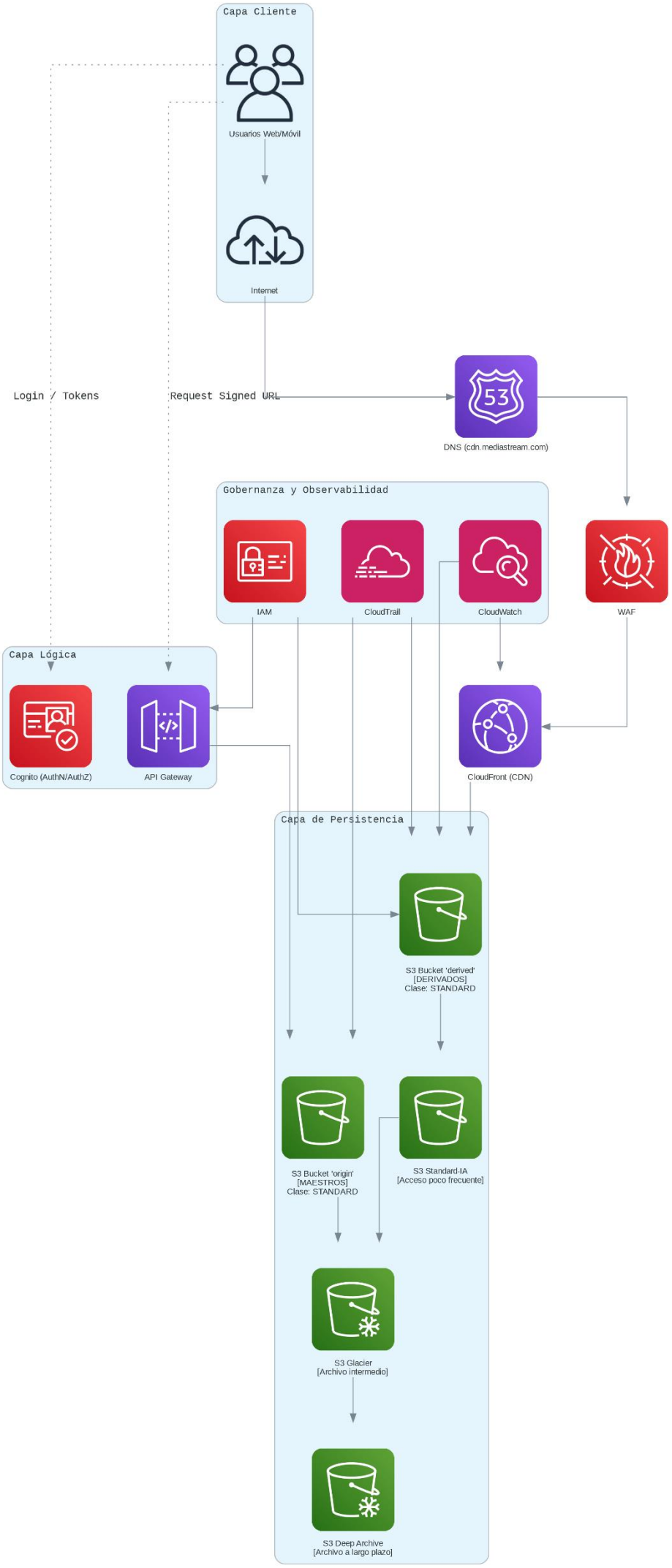
Integración de CDN y balanceo de carga

Servicios que NO cachea la CDN: autenticación, generación de URLs firmadas, APIs de catálogo/analytics. Hay que colocarlos detrás de un balanceador (ALB: application load balancer/API Gateway) con escalado horizontal automático del servicio (ECS/EKS/Lambda). La CDN entrega lo estático; el balanceador atiende lo dinámico; ambos se complementan.

Desacoplar cargas pesadas: Cuando un objeto llega al bucket, dispara funciones/procesos (resize, transcode, etiquetado) mediante eventos, esto mantiene la capa web liviana y escalable.

6) Propuesta Gráfica:

A continuación, se presenta un diagrama con la propuesta gráfica de la arquitectura planteada.


Capa Cliente
Usuarios Web/Móvil
Internet
Login / Tokens
Request Signed URL
DNS (cdn.mediastream.com)
Gobernanza y Observabilidad
IAM
CloudTrail
CloudWatch
WAF
Capa Lógica
Cognito (AuthN/AuthZ)
API Gateway
CloudFront (CDN)
Capa de Persistencia
S3 Bucket 'derived'
[DERIVADOS]
Clase: STANDARD
S3 Bucket 'origin'
[MAESTROS]
Clase: STANDARD
S3 Standard-IA
[Acceso poco frecuente]
S3 Glacier
[Archivo intermedio]
S3 Deep Archive
[Archivo a largo plazo]

Capa Cliente:

- **Usuarios Web/Móvil**: consumidores del contenido.
- Internet: red pública por donde viajan las peticiones.
- **Route53 (DNS):** resuelve el dominio cdn.mediastream.com hacia la distribución de CloudFront.
- **WAF (Web Application Firewall):** protege de ataques comunes.
- **CloudFront (CDN):** entrega el contenido con baja latencia desde el caché global en edge locations.

Usuarios → Internet → DNS (Route53) → WAF → CloudFront → S3 (derivados) con OAC

Los usuarios acceden por HTTPS al dominio público cdn.mediastream.com (Route53). WAF protege contra ataques maliciosos. CloudFront (CDN) entrega el contenido desde el caché global más cercano. Si el objeto no está en caché, CloudFront lo obtiene del bucket privado derived en S3 mediante OAC.

Capa lógica:

- **Cognito**: gestiona la autenticación (AuthN) y autorización (AuthZ) de usuarios administradores/colaboradores que suben contenido.
- **API Gateway**: expone un endpoint seguro para solicitar signed URLs.

Usuario autenticado → Cognito → API Gateway (request signed URL)

Un usuario productor se autentica con Cognito (login/tokens) y solicita una signed URL a la API. Esa URL permite subir directamente el archivo maestro al bucket origin en S3, sin pasar por servidores intermedios**.**

Capa de persistencia:

- **S3 origin (MAESTROS – Standard):**
    - Almacena los archivos originales (videos/audio en máxima calidad). Son privados y tienen versionado habilitado.
    - Flujo: recibe subidas vía signed URL.

- **S3 derived (DERIVADOS – Standard):**
    - Contiene las copias optimizadas para streaming. Es privado, accesible solo desde CloudFront por OAC.
    - Flujo: CloudFront obtiene de aquí los objetos que no tiene en

caché y los entrega al usuario.

- **S3 Standard-IA (Infrequent Access):**
  - Derivados menos consultados.

- **S3 Glacier y Deep Archive:**
  - Contienen contenido histórico o legalmente sensible (masters antiguos, backups completos). Son de muy bajo costo, con tiempos de recuperación más largos.

Las reglas de ciclo de vida van moviendo automáticamente los objetos a IA, Glacier o Deep Archive según su antigüedad y frecuencia de acceso.

Gobernanza y Monitoreo:

- **IAM:** define roles y políticas de acceso mínimos necesarios. Aplica permisos sobre API, S3 y CDN.
- **CloudWatch:** métricas de rendimiento (latencia, solicitudes, costos). Recolecta métricas de CDN y S3 para alertas.
- **CloudTrail:** auditoría de todas las operaciones sobre buckets y servicios. Registra quién accede o modifica qué datos, garantizando trazabilidad.

Conclusión

La solución propuesta para MediaStream se basa en una arquitectura escalable y segura, que aprovecha almacenamiento de objetos en múltiples clases de S3 y una CDN global (CloudFront) como único punto de entrega. Esta estrategia garantiza baja latencia, reducción de costos mediante políticas de ciclo de vida, y una gestión eficiente del contenido multimedia. El uso de URLs firmadas, IAM, CloudTrail y CloudWatch asegura control de acceso, trazabilidad y monitoreo continuo, permitiendo a la empresa responder a la creciente demanda internacional con alta disponibilidad y resiliencia.

7) Referencias

https://aws.amazon.com/es/cloudtrail/
https://aws.amazon.com/es/guardduty/
https://aws.amazon.com/es/config/
https://developer.hashicorp.com/terraform
https://aws.amazon.com/es/cloudformation/